

A computer system has four different types of resources (printer, ROM, hard disk and RAM) and it needs to complete execution of five processes (Google Drive, Firefox, Word Processor, Excel and PowerPoint)' The number of allocated resources, maximum needed resources to complete the execution and total available resources should be taken from the user. (Sample data for reference is given below). Calculate current need of resources by each process. Find whether these processes can be executed with available resources. If yes, show the correct order of process execution.

CODE:

```
GNU nano 6.2 Banker.c
#include <stdio.h>

int main() {
    int n = 5, m = 4; // 5 processes, 4 resources
    int alloc[5][4], max[5][4], need[5][4];
    int avail[4];
    int finish[5] = {0}, safeSeq[5];
    int i, j, k, count = 0;

    printf("Enter Allocation Matrix (5x4):\n");
    for(i=0; i<n; i++){
        for(j=0; j<m; j++){
            scanf("%d", &alloc[i][j]);
        }
    }

    printf("Enter Maximum Matrix (5x4):\n");
    for(i=0; i<n; i++){
        for(j=0; j<m; j++){
            scanf("%d", &max[i][j]);
        }
    }

    printf("Enter Available Resources (4):\n");
    for(i=0; i<m; i++){
        scanf("%d", &avail[i]);
    }

    // Calculate Need matrix
    for(i=0; i<n; i++){
        for(j=0; j<m; j++){
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }

    printf("\nNeed Matrix:\n");
    for(i=0; i<n; i++){
        for(j=0; j<m; j++){
            printf("%d ", need[i][j]);
        }
        printf("\n");
    }
}
```

```
who@Waghmare08: ~  
GNU nano 6.2 Banker.c  
    printf("\n");  
}  
  
// Banker's Algorithm  
while(count < n){  
    int found = 0;  
    for(i=0;i<n;i++){  
        if(finish[i]==0){  
            int flag = 0;  
            for(j=0;j<m;j++){  
                if(need[i][j] > avail[j]){  
                    flag = 1;  
                    break;  
                }  
            }  
  
            if(flag==0){  
                for(k=0;k<m;k++){  
                    avail[k] += alloc[i][k];  
                }  
                safeSeq[count++] = i;  
                finish[i] = 1;  
                found = 1;  
            }  
        }  
    }  
  
    if(found==0){  
        printf("\nSystem is NOT in safe state (Deadlock possible).\n");  
        return 0;  
    }  
}  
  
printf("\nSystem is in SAFE state.\nSafe sequence is:\n");  
  
for(i=0;i<n;i++){  
    printf("P%d ",safeSeq[i]);  
}  
  
printf("\n");  
return 0;  
}
```

OUTPUT:

```
who@Waghmare08: ~  
who@Waghmare08:~$ nano Banker.c  
who@Waghmare08:~$ gcc Banker.c  
who@Waghmare08:~$ ./a.out  
Enter Allocation Matrix (5x4):  
1 0 1 2  
2 1 0 1  
3 1 2 1  
0 2 1 0  
1 1 0 2  
Enter Maximum Matrix (5x4):  
3 2 1 2  
2 2 1 3  
3 3 2 2  
1 2 2 1  
2 2 1 3  
Enter Available Resources (4):  
3 3 2 2  
  
Need Matrix:  
2 2 0 0  
0 1 1 2  
0 2 0 1  
1 0 1 1  
1 1 1 1  
  
System is in SAFE state.  
Safe sequence is:  
P0 P1 P2 P3 P4  
who@Waghmare08:~$ |
```